Hoàng Trí Nhật
N25DECE083
E25CQCE02-N

Problem 1: Singly linked list - In-place reversal & Pointer Tracing

Given a singly linked list: head → [10] → [25] → [40] → [15] → NULL

- Task 1: Write a function/algorith in pseudocode to reverse the singly linked list, including three pointers (prev, curr, next) to complete your function
- Task 2: Draw a diagram or step-by-step pointer trace table showing the position and values of the three pointers (prev, curr, next) after each iteration until the list is fully reversed

Answer:

Task 1:

```
Function Reverse (head):
    prev ← NULL
    curr ← head
    next ← NULL
    While curr ≠ NULL:
        next ← curr.next
        curr.next ← prev
        prev ← next
        curr ← next

    head ← prev
    Return head
```

VIBOOK INTERNATIONAL

Scanned with CamScanner

Task 2:

Iteration	prev	curr	next
Initial	NULL	10	-
1	10	25	25
2	25	40	40
3	40	15	15
4	15	NULL	NULL

Final list: head -> [15] -> [40] -> [25] -> [10] -> NULL

Problem 2: Circular linked list - Insert at Head

Given a singly circular linked list: head -> [10] -> [20] -> [30] -> (points back to head)

- Task 1: Write pseudocode for the function InsertHead(head, newValue) to insert a new node containing newValue at the front of circular linked list while maintaining its circular structure.
- Task 2: Trace the execution steps (via text description or pointer diagrams) when insert newValue = 5 into the given list

Answer:

Task 1:

```
Function InsertHead(head, newValue):

    newNode ← CreateNode(newValue)

    If head = NULL:
        newNode.next ← newNode
        head ← newNode
        Return head
```

VIBOOK INTERNATIONAL

Scanned with CamScanner

```
curr ← head

while (curr.next ≠ head):
    curr ← curr.next

newNode.next ← head
curr.next ← newNode
head ← newNode

Return head
```

10) Task 7 :

Initial :

head → [10] → [20] → [30]

Create Node* newNode = new Node(5);

Steps :

1. curr = 10
2. curr = 20
3. curr = 30
   → 30.next = head, so 30 is the last node
4. newNode.next = head
5. 30.next = newNode
6. head = newNode

Final :

head → [5] → [10] → [20] → [30]

VIBOOK INTERNATIONAL

Scanned with CamScanner

2.5 Problem 3:

Given a doubly linked list containing integers

- Task 1: Write pseudocode for InsertBefore(head, target, newValue) to insert a new node containing newValue immediately before the first occurrence of target
- Task 2: List the exact order of pointer assignments required and briefly justify why this specific sequence avoids memory leak or broken links
- Task 3: Explicitly handle the following two edge cases in code/pseudocode:
  1. target is located at the head node
  2. target does not exist in the list

Answer:

0.75 Task 1:

```
Function InsertBefore(head, target, newValue):
    curr ← head
    While (curr ≠ NULL AND curr.value ≠ target):
        curr ← curr.next
    If curr = NULL:
        Return head
    newNode ← CreateNode(newValue)
    newNode.next ← curr
    newNode.pre ← curr.pre
    If (curr.prev ≠ NULL):
        curr.prev.next ← newNode
    Else: head ← newNode
    curr.prev ← newNode
    Return head
```

VIBOOK INTERNATIONAL

Scanned with CamScanner

Hoang Trí Nhân
N25DECE083
E25CQCE02-N

Task 2:

1. newNode.next ← curr
2. newNode.pre ← curr.~~next~~prev
3. If curr.prev ≠ NULL:
   curr.prev.next ← newNode
4. curr.pre ← newNode

Justification: This order preserves the original links before changing them, preventing broken links and losing access to nodes.

10 Task 3:

1. Target is head:
If curr.prev = NULL:
   head ← newNode

Then:
   curr.pre ← newNode

2. Target does not exist:
If curr = NULL:
   Return head

25 Problem 4:

Given a singly linked list sorted in ascending order:
head → [2] → [5] → [8] → NULL

- Task 1: Write the function SortedInsert(head, val) to insert

VIBOOK INTERNATIONAL

Scanned with CamScanner

val into the list such that the ascending order is preserved.

- Task 2: Construct a step-by-step trace table detailing pointer iterations and condition evaluation when inserting val = 6 into the list

Answer

Task 1:

```
Function SortedInsert(head, val):
    newNode ← CreateNode(val)
    If head = NULL OR val < head.value:
        newNode.next ← head
        head ← newNode
        Return head
    curr ← head
    While curr.next ≠ NULL AND curr.next.value < val:
        curr ← curr.next
    newNode.next ← curr.next
    curr.next ← newNode
    Return head
```

Task 2:

Step	curr	curr.next	Condition	Action
Initial	2	5	6 < 2 → False	Not insert a head
1	2	5	5 < 6 → True	curr = 5
2	5	8	8 < 6 → False	Stop
Insert	5	8	-	Insert 6

VIBOOK INTERNATIONAL

Scanned with CamScanner

Final list :

head -> [2] -> [5] -> [6] -> [9] -> NULL

VIBOOK
INTERNATIONAL

Scanned with CamScanner